

Log- Shift-Operationen und Casting

Aufgabe 1 – Logische Instruktionen

Geben Sie die Assemblerinstruktionen für die folgenden Fälle an.

- a) Invertieren des Inhaltes des Registers R1 (Bilden des Einerkomplementes)

- b) Verändern Sie den Inhalt des Registers R1 so, dass
 - die Bits 3..0 alle eins,
 - die Bits 7..4 alle null,
 - die Bits 17-16 alle invertiert und
 - alle übrigen Bits unverändert sind.

Aufgabe 2 – Multiplikation mit einer Konstanten

Schreiben Sie eine Codesequenz in Assembler welche den vorzeichenlosen Inhalt des Registers R7 mit der Dezimalzahl 43 multipliziert und im Register R7 abspeichert.

- a) Verwenden Sie die Multiplikationsinstruktion

- b) Ersetzen Sie die Multiplikationsinstruktion durch Shift- und Additionsbefehle

Aufgabe 3 – Reverse Engineering

Die folgenden Assemblersequenzen wurden durch den bcc Compiler aus einem C Programm erzeugt. Schreiben Sie einen möglichen C-Code, welcher diese Assemblersequenz erzeugt.

Die Speicherstellen im Assembler werden den ursprünglichen Variablen im C-Programm wie folgt zugeordnet:

```

ux:    .word ?      ↔    uint32_t ux
uy:    .word ?      ↔    uint32_t uy
uz:    .word ?      ↔    uint32_t uz

```

	Assembler	C
Beispiel	<pre> ldr r0,=ux ldr r1,[r0] adds r1,#1 str r1,[r0] </pre>	<pre> ux++; </pre>
a)	<pre> ldr r0,=ux ldr r1,[r0] lsls r1,r1,#1 ldr r2,=uy ldr r3,[r2] adds r3,r1 lsls r3,r3,#3 str r3,[r2] </pre>	
b)	<pre> ldr r0,=ux ldr r1,[r0] ldr r2,=uy ldr r3,[r2] ldr r4,=uz ldr r5,[r4] lsrs r1,r1,#3 lsls r3,r3,#4 orrs r1,r3 mvns r1,r1 ands r1,r5 str r1,[r4] </pre>	

Aufgabe 4 – Explicit Casting in C

- a) Gegeben ist der folgende C Code:

```
uint8_t ux = 100;  
int8_t  sx = (int8_t)ux;
```

Als welche Dezimalzahl wird der Inhalt der Variable sx nach dem Cast interpretiert?

- b) Gegeben ist der folgende C Code:

```
int8_t  sx = -10;  
uint8_t ux = (uint_8)sx;
```

Als welche Dezimalzahl wird der Inhalt der Variable ux nach dem Cast interpretiert?